Code Deployment vs Configuration

The CNP platform is built for software engineers to build and professional services (configuration experts) to operate.

ENGINEERING
Workflow Engine (state machine executor)
Rules Evaluator (named hook implementations)
Configuration API (the boundary surface)
OIDC Auth Broker + IAM engine
UI Framework + component library
Audit / snapshot infrastructure

⟵ APIs ⟶

CONFIGURATION
State workflow overrides (TenantTransitionOverrides)
CnpDefaults per state (TenantConfigOverrides)
Feature flag overrides (TenantFlagOverrides)
Role and permission assignments (per state)
Notification templates and routing
DC feed adapter configurations
Accounting export adapter configurations
New state onboarding (config scaffold)

Configuration APIs

These are the API endpoints onshore teams use to configure the platform. Every endpoint requires an authenticated onshore admin JWT. All writes are logged to FieldChangeLog. No

endpoint allows overriding a federal-locked value.

CnpDefaults API

Method	Endpoint	Purpose
GET	`/config/definitions?category=Claims`	List all config keys in a category with defaults, bounds, federal status
GET	`/config/{tenant}/overrides`	See all active overrides for a state
POST	`/config/{tenant}/overrides`	Set or future-date a state-specific value
DELETE	`/config/{tenant}/overrides/{id}`	Remove an override (reverts to platform default)
GET	`/config/{tenant}/resolve?key=X&program=Y`	Preview what value the engine will return — useful for verifying before cutover

Workflow Override API

Method	Endpoint	Purpose
GET	`/workflow/{entityType}/transitions`	See all base transitions with federal status for an entity type
GET	`/workflow/{tenant}/{entityType}/effective`	See the effective (base + overrides applied) transition map for a tenant
POST	`/workflow/{tenant}/overrides`	Add, disable, or modify a transition for a state — guardrail-checked at write
GET	`/workflow/hooks`	List all available named condition and action hooks (read-only catalog)

Feature Flag API

Method	Endpoint	Purpose
GET	`/flags`	List all flag definitions with defaults
GET	`/flags/{tenant}`	See all flag states for a tenant
PUT	`/flags/{tenant}/{flagKey}`	Enable or disable a module for a state

DC Feed Adapter API

Method	Endpoint	Purpose
GET	`/dc-feeds/{tenant}`	List configured DC feed adapters for a state
POST	`/dc-feeds/{tenant}`	Register a new feed adapter (SFTP, REST, file) with credentials stored in Secrets Manager
PUT	`/dc-feeds/{tenant}/{feedId}/schedule`	Set the nightly load schedule for a feed
POST	`/dc-feeds/{tenant}/{feedId}/test`	Trigger a test connection to verify credentials before going live

Accounting Export Adapter API

Each state agency that transmits payments to a government accounting system requires a unique `IAccountingExporter` implementation. This API registers and operates those adapters. The adapter identified by `adapterKey` must already be deployed before registration — see Accounting Export Adapter API for the full reference and adapter registry.

Method	Endpoint	Purpose

GET	`/accounting-adapters/{tenant}`	List registered accounting export adapters for a state
POST	`/accounting-adapters/{tenant}`	Register an adapter for a state using its `adapterKey` (e.g. STAARS, SABHRS, GAX_CO)
PUT	`/accounting-adapters/{tenant}/{adapterId}/schedule`	Set the automated export schedule for an adapter
POST	`/accounting-adapters/{tenant}/{adapterId}/test`	Test the adapter configuration before going live
POST	`/accounting-adapters/{tenant}/{adapterId}/run`	Trigger an ad-hoc export run
GET	`/accounting-adapters/{tenant}/{adapterId}/batches`	List export batch history for audit and troubleshooting

Tenant Onboarding API

Method	Endpoint	Purpose
POST	`/tenants`	Register a new state tenant — creates cnp_ops schema, seeds platform defaults
POST	`/tenants/{tenant}/scaffold?similarTo=UT`	AI-generated starter config based on similarity to an existing state
GET	`/tenants/{tenant}/readiness`	Checklist of required config items not yet set — blocks go-live until resolved

Configuration vs Deployment

Configuration

Action	How it's done	API used
Enable a module for a new state	Set TenantFlagOverride	Feature Flag API
Change a state's late claim cutoff	POST TenantConfigOverride with EffectiveDate	CnpDefaults API
Add Maine's auto-decline behavior	POST TenantTransitionOverride (AddCondition)	Workflow Override API
Add a new DC feed for a state (e.g. new Medicaid endpoint)	POST dc-feeds with credentials	DC Feed Adapter API
Register an accounting export adapter for a state	POST /accounting-adapters/{tenant} with adapterKey — adapter must already be deployed	Accounting Export Adapter API
Onboard a new state	POST /tenants + scaffold + configure overrides	Tenant Onboarding API
Change a notification reminder schedule	Update TenantConfigOverride for Notifications.ClaimReminderDays	CnpDefaults API
Update annual reimbursement rates	INSERT into ClaimRateAmounts in cnp_global	Rate Management API (Phase 2)
Future-date a config change for July 1	POST TenantConfigOverride with EffectiveDate=2026-07-01	CnpDefaults API
Grant a new role permission to trigger a transition	POST WorkflowRolePermission for tenant	Workflow Override API

Enable Session Replay for a specific state	PUT TenantFlagOverride + POST SessionReplayConfig	Feature Flag API

Deployment

Action	Why it needs a deployment
Add a new named condition hook (e.g. a new validation rule type)	Code implementation required in the condition evaluator
Add a new named action hook (e.g. a new system integration)	Code implementation required in the action executor
Add a new module (e.g. a new federal program)	Domain model, business logic, and UI components need to be built
Change any federal math formula	Federal math lives in code hooks, not configuration — by design
Add a new ConfigDefinition key	New config keys are seeded via database migration, not the admin UI
Change IsFederal on any row	Requires migration review — cannot be done from admin UI
Add a new WorkflowState to an entity type	Data migration in cnp_global; may require UI changes
Integrate a new external system (new Ed-Fi adapter, new accounting export)	New IEdFiAdapter or IAccountingExporter implementation

Anti-pattern: Hardcoding items for states because the Configuration API doesn't support it yet.

Solution: Add the missing API surface then configure through it. Hardcoding state-specific logic in application code is exactly what this design is meant to eliminate.

Onboarding a New State Agency

The test of the boundaries described above is: Can we onboard a new state agency without a code deployment? The answer should be yes for everything except truly novel requirements.

Step	How

Register the new tenant	`POST /tenants {tenantCode: "WI"}` — creates cnp_ops schema, seeds all platform defaults
Generate AI starter config	`POST /tenants/WI/scaffold?similarTo=MN` — AI suggests overrides based on similar states
Review and approve scaffold	Review each suggested override, approve or modify in admin UI
Enable modules	Set TenantFlagOverrides for SNP, CACFP, SFSP, SSFO as applicable
Configure DC feeds	Register SNAP and Medicaid feed adapters with credentials via DC Feed API
Register accounting export adapter	`POST /accounting-adapters/{tenant}` with the state's adapterKey. Confirm the adapter is deployed first — if not, a code deployment is required before this step. See Accounting Export Adapter API for the full registry.
Set workflow overrides	Configure any state-specific transition rules (auto-decline, DirectVer, etc.)
Validate readiness	`GET /tenants/WI/readiness` — returns checklist of unresolved required config
Run data replay validation	Replay existing WI data through new engine, compare outputs
Go-live cutover	Enable production flag, decommission old connection

Governance Model

API contract ownership

The Configuration API contract is owned by Product and Engineering jointly. Changes to the API surface (new endpoints, new parameters, changed validation rules) require sign-off from both.

Engineering cannot remove or break a Configuration API endpoint without communication and approval. The configuration team cannot request a new API surface without a written use case that Product and Engineering accepts.

Definition of Done

Each phase is not complete until the configuration team can accomplish everything in that phase's scope through the Configuration API without a code change.